

List in Python (Detailed Explanation)

A **List** is a collection of items stored in a single variable.

Features of Lists

- Ordered (items have indexes)
 - Mutable (can be changed)
 - Allows duplicate values
 - Can store different data types
-

Creating a List

```
fruits = ["Apple", "Banana", "Mango"]  
print(fruits)
```

Output:

```
['Apple', 'Banana', 'Mango']
```

List with Different Data Types

```
data = [10, "Python", 3.14, True]  
print(data)
```

Output:

```
[10, 'Python', 3.14, True]
```

Accessing List Elements

Lists use **indexing**.

```
fruits = ["Apple", "Banana", "Mango"]  
  
print(fruits[0])  
print(fruits[1])  
print(fruits[2])
```

Output:

```
Apple  
Banana  
Mango
```

Negative Indexing

```
fruits = ["Apple", "Banana", "Mango"]  
  
print(fruits[-1])  
print(fruits[-2])
```

Output:

```
Mango  
Banana
```

Slicing Lists

Syntax

```
list[start:stop:step]
```

Example

```
numbers = [10, 20, 30, 40, 50]  
  
print(numbers[1:4])
```

Output:

```
[20, 30, 40]
```

Modifying List Elements

```
fruits = ["Apple", "Banana", "Mango"]  
  
fruits[1] = "Orange"  
  
print(fruits)
```

Output:

```
['Apple', 'Orange', 'Mango']
```

Adding Elements

1. append()

Adds one element at the end.

```
fruits = ["Apple", "Banana"]  
  
fruits.append("Mango")  
  
print(fruits)
```

Output:

```
['Apple', 'Banana', 'Mango']
```

2. insert()

Adds an element at a specific index.

```
fruits = ["Apple", "Banana"]  
  
fruits.insert(1, "Orange")  
  
print(fruits)
```

Output:

```
['Apple', 'Orange', 'Banana']
```

3. extend()

Adds multiple elements.

```
fruits = ["Apple", "Banana"]  
  
fruits.extend(["Mango", "Orange"])  
  
print(fruits)
```

Output:

```
['Apple', 'Banana', 'Mango', 'Orange']
```

Removing Elements

1. remove()

Removes a specific value.

```
fruits = ["Apple", "Banana", "Mango"]  
  
fruits.remove("Banana")
```

```
print(fruits)
```

Output:

```
['Apple', 'Mango']
```

2. pop()

Removes element by index.

```
fruits = ["Apple", "Banana", "Mango"]  
  
fruits.pop(1)  
  
print(fruits)
```

Output:

```
['Apple', 'Mango']
```

3. del

```
fruits = ["Apple", "Banana", "Mango"]  
  
del fruits[0]  
  
print(fruits)
```

Output:

```
['Banana', 'Mango']
```

4. clear()

Removes all elements.

```
fruits = ["Apple", "Banana"]  
  
fruits.clear()  
  
print(fruits)
```

Output:

```
[]
```

List Methods

Method	Description
<code>append()</code>	Add item at end
<code>insert()</code>	Add item at index
<code>extend()</code>	Add multiple items
<code>remove()</code>	Remove specific item
<code>pop()</code>	Remove by index
<code>clear()</code>	Remove all items
<code>sort()</code>	Sort list
<code>reverse()</code>	Reverse list
<code>count()</code>	Count occurrences
<code>index()</code>	Find index
<code>copy()</code>	Copy list

Sorting a List

```
numbers = [40, 10, 30, 20]

numbers.sort()

print(numbers)
```

Output:

```
[10, 20, 30, 40]
```

Descending Order

```
numbers.sort(reverse=True)
print(numbers)
```

Output:

```
[40, 30, 20, 10]
```

Reverse a List

```
numbers = [1, 2, 3, 4]

numbers.reverse()

print(numbers)
```

Output:

```
[4, 3, 2, 1]
```

Count Elements

```
numbers = [1, 2, 2, 3, 2]
print(numbers.count(2))
```

Output:

3

Find Index

```
fruits = ["Apple", "Banana", "Mango"]
print(fruits.index("Banana"))
```

Output:

1

Copy a List

```
list1 = [1, 2, 3]
list2 = list1.copy()
print(list2)
```

Output:

[1, 2, 3]

Loop Through a List

```
fruits = ["Apple", "Banana", "Mango"]
for fruit in fruits:
    print(fruit)
```

Output:

Apple
Banana
Mango

List Comprehension

A short way to create lists.

```
numbers = [x*x for x in range(1, 6)]  
  
print(numbers)
```

Output:

```
[1, 4, 9, 16, 25]
```

Nested List

```
matrix = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
  
print(matrix[1][2])
```

Output:

```
6
```

Useful Built-in Functions

```
numbers = [10, 20, 30, 40]  
  
print(len(numbers))  
print(max(numbers))  
print(min(numbers))  
print(sum(numbers))
```

Output:

```
4  
40  
10  
100
```

Interview Questions

1. Difference between `append()` and `extend()`

```
a = [1, 2]
```

```
a.append([3, 4])
print(a)
```

Output:

```
[1, 2, [3, 4]]
a = [1, 2]
```

```
a.extend([3, 4])
print(a)
```

Output:

```
[1, 2, 3, 4]
```

2. Difference between `remove()` and `pop()`

- `remove(value)` → removes by value.
 - `pop(index)` → removes by index and returns the removed element.
-

Summary

Create List	-> []
Access Element	-> list[index]
Add Item	-> append(), insert(), extend()
Remove Item	-> remove(), pop(), del, clear()
Sort List	-> sort()
Reverse List	-> reverse()
Count Item	-> count()
Find Index	-> index()
Copy List	-> copy()
Loop List	-> for loop

Example Program

```
students = ["Krish", "Rahul", "Amit"]
```

```
students.append("Priya")
students.remove("Rahul")
```

```
for student in students:
    print(student)
```

Output:

```
Krish
Amit
Priya
```

Link : CHATGPT

List Comprehension in Python (Detailed Explanation)

List Comprehension એ Python માં List બનાવવાની એક short, fast અને readable રીત છે.

Basic Syntax

```
new_list = [expression for item in iterable]
```

Example

```
numbers = [1, 2, 3, 4, 5]
square = [x**2 for x in numbers]
print(square)
```

Output:

```
[1, 4, 9, 16, 25]
```

How it works

```
square = []
for x in numbers:
    square.append(x**2)
```

આ બંને code જ output એકસરખું છે.

Example 1: Create a List of Numbers

```
numbers = [x for x in range(1, 11)]
print(numbers)
```

Output

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Example 2: Square of Numbers

```
square = [x*x for x in range(1, 6)]  
print(square)
```

Output

```
[1, 4, 9, 16, 25]
```

Example 3: Cube of Numbers

```
cube = [x**3 for x in range(1, 6)]  
print(cube)
```

Output

```
[1, 8, 27, 64, 125]
```

List Comprehension with Condition

Syntax

```
new_list = [expression for item in iterable if condition]
```

Example 4: Even Numbers

```
even = [x for x in range(1, 11) if x % 2 == 0]  
print(even)
```

Output

```
[2, 4, 6, 8, 10]
```

Example 5: Odd Numbers

```
odd = [x for x in range(1, 11) if x % 2 != 0]  
print(odd)
```

Output

```
[1, 3, 5, 7, 9]
```

Using if-else in List Comprehension

Syntax

```
[true_value if condition else false_value for item in iterable]
```

Example 6

```
result = ["Even" if x % 2 == 0 else "Odd"
          for x in range(1, 6)]

print(result)
```

Output

```
['Odd', 'Even', 'Odd', 'Even', 'Odd']
```

Example 7: Positive and Negative

```
numbers = [-2, 4, -5, 7, -1]

result = ["Positive" if x > 0 else "Negative"
          for x in numbers]

print(result)
```

Output

```
['Negative', 'Positive', 'Negative', 'Positive', 'Negative']
```

String Example

Example 8: Convert to Uppercase

```
names = ["krish", "rahul", "amit"]

upper_names = [name.upper() for name in names]

print(upper_names)
```

Output

```
['KRISH', 'RAHUL', 'AMIT']
```

Example 9: Length of Each Word

```
words = ["Python", "Java", "C++"]  
  
lengths = [len(word) for word in words]  
  
print(lengths)
```

Output

```
[6, 4, 3]
```

Nested List Comprehension

Example 10

```
matrix = [[1, 2], [3, 4], [5, 6]]  
  
flat = [num for row in matrix for num in row]  
  
print(flat)
```

Output

```
[1, 2, 3, 4, 5, 6]
```

Equivalent For Loop

```
flat = []  
  
for row in matrix:  
    for num in row:  
        flat.append(num)
```

Creating a Multiplication Table

```
table = [5 * x for x in range(1, 11)]  
  
print(table)
```

Output

```
[5, 10, 15, 20, 25, 30, 35, 40, 45, 50]
```

Create First Letter List

```
names = ["Krish", "Rahul", "Amit"]
```

```
first_letter = [name[0] for name in names]

print(first_letter)
```

Output

```
['K', 'R', 'A']
```

Remove Spaces

```
text = "Python Programming"

letters = [ch for ch in text if ch != " "]

print(letters)
```

Output

```
['P', 'y', 't', 'h', 'o', 'n', 'P', 'r', 'o', 'g', 'r', 'a', 'm', 'm', 'i',
'n', 'g']
```

Advantages of List Comprehension

- ✓ Short code
 - ✓ Easy to read
 - ✓ Faster than normal loops
 - ✓ Pythonic way of writing code
-

When NOT to Use List Comprehension

- ✗ Very complex logic
- ✗ Multiple nested conditions
- ✗ When readability becomes difficult

Bad Example:

```
result = [x**2 if x%2==0 else x**3
           for x in range(20)]
```

```
if x > 5]
```

આવલ cases માં સામાન્ય for loop વધુ readable હોય છે.

Most Important Interview Examples

Squares

```
[x**2 for x in range(1, 11)]
```

Even Numbers

```
[x for x in range(1, 21) if x % 2 == 0]
```

Odd Numbers

```
[x for x in range(1, 21) if x % 2 != 0]
```

Uppercase Names

```
[name.upper() for name in names]
```

First Characters

```
[name[0] for name in names]
```

Flatten Nested List

```
[num for row in matrix for num in row]
```

Formula

```
[expression for item in iterable]  
[expression for item in iterable if condition]  
[true_value if condition else false_value for item in iterable]
```

Link : CHATGPT

Set in Python (Detailed Explanation)

A **Set** is a built-in data structure in Python used to store a collection of **unique elements**.

A set is useful when you want to remove duplicates or perform mathematical set operations like union and intersection.

What is a Set?

A set is:

- ✓ Unordered
 - ✓ Mutable (can add/remove elements)
 - ✓ Does not allow duplicate values
 - ✓ Can contain different data types
 - ✗ Does not support indexing or slicing
-

Creating a Set

Method 1: Using Curly Braces {}

```
fruits = {"Apple", "Banana", "Mango"}  
print(fruits)
```

Output

```
{'Apple', 'Banana', 'Mango'}
```

Method 2: Using `set()`

```
numbers = set([1, 2, 3, 4])  
print(numbers)
```

Output

```
{1, 2, 3, 4}
```

Empty Set

⚠ Important

```
s = {}  
print(type(s))
```

Output:

```
<class 'dict'>
```

`{}` creates an empty dictionary, not a set.

To create an empty set:

```
s = set()  
print(type(s))
```

Output:

```
<class 'set'>
```

Duplicate Values

Sets automatically remove duplicates.

```
numbers = {1, 2, 2, 3, 4, 4, 5}  
print(numbers)
```

Output

```
{1, 2, 3, 4, 5}
```

Accessing Set Elements

Sets do not support indexing.

❌ Invalid:

```
fruits = {"Apple", "Banana", "Mango"}  
print(fruits[0])
```

Output:

```
TypeError: 'set' object is not subscriptable
```

Use a loop instead:

```
for fruit in fruits:  
    print(fruit)
```

Adding Elements

1. `add()`

Adds one element.

```
fruits = {"Apple", "Banana"}  
  
fruits.add("Mango")  
  
print(fruits)
```

Output:

```
{'Apple', 'Banana', 'Mango'}
```

2. `update()`

Adds multiple elements.

```
fruits = {"Apple", "Banana"}  
  
fruits.update(["Mango", "Orange"])  
  
print(fruits)
```

Output:

```
{'Apple', 'Banana', 'Mango', 'Orange'}
```

Removing Elements

1. `remove()`

Removes a specific element.

```
fruits = {"Apple", "Banana"}
```

```
fruits.remove("Banana")  
  
print(fruits)
```

Output:

```
{'Apple'}
```

If the element doesn't exist:

```
fruits.remove("Mango")
```

Output:

```
KeyError
```

2. `discard()`

Removes an element without error.

```
fruits = {"Apple", "Banana"}  
  
fruits.discard("Mango")  
  
print(fruits)
```

Output:

```
{'Apple', 'Banana'}
```

3. `pop()`

Removes and returns a random element.

```
fruits = {"Apple", "Banana", "Mango"}  
  
item = fruits.pop()  
  
print(item)  
print(fruits)
```

Example Output

```
Banana  
{'Apple', 'Mango'}
```

⚠ Since sets are unordered, the removed element can vary.

4. `clear()`

Removes all elements.

```
fruits.clear()
print(fruits)
```

Output:

```
set()
```

Set Methods

Method	Description
<code>add()</code>	Add one element
<code>update()</code>	Add multiple elements
<code>remove()</code>	Remove element
<code>discard()</code>	Remove without error
<code>pop()</code>	Remove random element
<code>clear()</code>	Remove all elements
<code>copy()</code>	Copy set

Membership Operators

```
fruits = {"Apple", "Banana", "Mango"}
print("Banana" in fruits)
print("Orange" in fruits)
```

Output:

```
True
False
```

Loop Through a Set

```
fruits = {"Apple", "Banana", "Mango"}
for fruit in fruits:
    print(fruit)
```

Copying a Set

```
s1 = {1, 2, 3}
s2 = s1.copy()
print(s2)
```

Output:

```
{1, 2, 3}
```

Mathematical Set Operations

Suppose:

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
```

1. Union

Combines all unique elements.

```
print(A.union(B))
```

or

```
print(A | B)
```

Output:

```
{1, 2, 3, 4, 5, 6}
```

2. Intersection

Common elements.

```
print(A.intersection(B))
```

or

```
print(A & B)
```

Output:

```
{3, 4}
```

3. Difference

Elements present in the first set only.

```
print(A.difference(B))
```

or

```
print(A - B)
```

Output:

```
{1, 2}
```

4. Symmetric Difference

Elements present in either set but not both.

```
print(A.symmetric_difference(B))
```

or

```
print(A ^ B)
```

Output:

```
{1, 2, 5, 6}
```

Subset and Superset

Subset

```
A = {1, 2}
B = {1, 2, 3, 4}
```

```
print(A.issubset(B))
```

Output:

```
True
```

Superset

```
print(B.issuperset(A))
```

Output:

```
True
```

Disjoint Sets

Checks whether sets have no common elements.

```
A = {1, 2}
B = {3, 4}

print(A.isdisjoint(B))
```

Output:

```
True
```

Frozenset

A frozenset is an immutable set.

```
fs = frozenset([1, 2, 3])

print(fs)
```

Output:

```
frozenset({1, 2, 3})
```

Cannot modify it:

```
fs.add(4)
```

Output:

```
AttributeError
```

Useful Built-in Functions

```
numbers = {10, 20, 30, 40}

print(len(numbers))
print(max(numbers))
print(min(numbers))
print(sum(numbers))
```

Output:

4
40
10
100

List vs Tuple vs Dictionary vs Set

Feature	List	Tuple	Dictionary	Set
Syntax	[]	()	{key:value}	{}
Ordered	Yes	Yes	Yes	No
Mutable	Yes	No	Yes	Yes
Duplicates	Yes	Yes	Keys: No	No
Indexing	Yes	Yes	By Key	No

Real-Life Example

Remove duplicate names:

```
names = ["Krish", "Rahul", "Krish", "Amit", "Rahul"]  
unique_names = set(names)  
print(unique_names)
```

Output:

```
{'Krish', 'Rahul', 'Amit'}
```

Interview Questions

1. Can sets contain duplicates?

✗ No.

```
s = {1, 1, 2, 2}  
print(s)
```

Output:

```
{1, 2}
```

2. Can sets be indexed?

✗ No.

Sets are unordered.

3. Difference between `remove()` and `discard()`?

- `remove()` → Raises `KeyError` if element doesn't exist.
 - `discard()` → Does not raise an error.
-

4. What is the difference between `set` and `frozenset`?

<code>set</code>	<code>frozenset</code>
Mutable	Immutable
Can add/remove	Cannot modify

Summary

Create Set	-> <code>{}</code>
Empty Set	-> <code>set()</code>
Add Element	-> <code>add()</code>
Add Multiple	-> <code>update()</code>
Remove	-> <code>remove()</code> , <code>discard()</code> , <code>pop()</code> , <code>clear()</code>
Membership	-> <code>in</code>
Union	-> <code>union()</code> or <code> </code>
Intersection	-> <code>intersection()</code> or <code>&</code>
Difference	-> <code>difference()</code> or <code>-</code>
Symmetric Diff	-> <code>symmetric_difference()</code> or <code>^</code>
Subset	-> <code>issubset()</code>
Superset	-> <code>issuperset()</code>
Disjoint	-> <code>isdisjoint()</code>
Immutable Set	-> <code>frozenset()</code>

Example Program

```
A = {1, 2, 3}
B = {3, 4, 5}

print("Union:", A | B)
print("Intersection:", A & B)
print("Difference:", A - B)
print("Symmetric Difference:", A ^ B)
```

Output

```
Union: {1, 2, 3, 4, 5}
```

Intersection: {3}
Difference: {1, 2}
Symmetric Difference: {1, 2, 4, 5}

Important Points to Remember

- **Set = Collection of unique elements**
- **No duplicates allowed**
- **No indexing or slicing**
- **Best for removing duplicates and performing mathematical set operations**
- **frozenset is an immutable version of a set**

Link : CHATGPT

Tuple in Python (Detailed Explanation)

A **Tuple** is a collection of items stored in a single variable.

Tuple is similar to a List, but the main difference is:

- ✅ **List is mutable** (can be changed)
 - ❌ **Tuple is immutable** (cannot be changed)
-

Creating a Tuple

Example

```
fruits = ("Apple", "Banana", "Mango")
print(fruits)
```

Output:

```
('Apple', 'Banana', 'Mango')
```

Characteristics of Tuple

- Ordered
 - Immutable
 - Allows duplicate values
 - Supports indexing and slicing
 - Faster than lists
-

Tuple with Different Data Types

```
data = (10, "Python", 3.14, True)
print(data)
```

Output:

```
(10, 'Python', 3.14, True)
```

Accessing Tuple Elements

Positive Indexing

```
fruits = ("Apple", "Banana", "Mango")

print(fruits[0])
print(fruits[1])
print(fruits[2])
```

Output:

```
Apple
Banana
Mango
```

Negative Indexing

```
fruits = ("Apple", "Banana", "Mango")

print(fruits[-1])
print(fruits[-2])
```

Output:

```
Mango
Banana
```

Slicing a Tuple

```
numbers = (10, 20, 30, 40, 50)

print(numbers[1:4])
```

Output:

```
(20, 30, 40)
```

Tuple Length

```
numbers = (10, 20, 30, 40)

print(len(numbers))
```

Output:

```
4
```

Single Element Tuple

⚠ Important

```
t = (5)
print(type(t))
```

Output:

```
<class 'int'>
```

To create a tuple with one element, use a comma:

```
t = (5,)
print(type(t))
```

Output:

```
<class 'tuple'>
```

Tuple Packing

```
student = ("Krish", 20, "Rajkot")
print(student)
```

Python automatically packs values into a tuple.

Tuple Unpacking

```
student = ("Krish", 20, "Rajkot")
name, age, city = student
print(name)
print(age)
print(city)
```

Output:

```
Krish
20
Rajkot
```

Loop Through a Tuple

Using for Loop

```
fruits = ("Apple", "Banana", "Mango")

for fruit in fruits:
    print(fruit)
```

Output:

```
Apple
Banana
Mango
```

Checking an Element

Using in

```
fruits = ("Apple", "Banana", "Mango")

print("Banana" in fruits)
```

Output:

```
True
```

Tuple Methods

Tuple has only **2 built-in methods**.

Method	Description
count()	Counts occurrences
index()	Returns index of value

count()

```
numbers = (1, 2, 2, 3, 2)

print(numbers.count(2))
```

Output:

```
3
```

index()

```
fruits = ("Apple", "Banana", "Mango")
```

```
print(fruits.index("Banana"))
```

Output:

```
1
```

Cannot Modify Tuple

```
fruits = ("Apple", "Banana", "Mango")  
fruits[1] = "Orange"
```

Output:

```
TypeError: 'tuple' object does not support item assignment
```

Because tuples are immutable.

Convert Tuple to List

```
fruits = ("Apple", "Banana", "Mango")  
temp = list(fruits)  
temp.append("Orange")  
fruits = tuple(temp)  
print(fruits)
```

Output:

```
('Apple', 'Banana', 'Mango', 'Orange')
```

Concatenation

```
t1 = (1, 2, 3)  
t2 = (4, 5, 6)  
t3 = t1 + t2  
print(t3)
```

Output:

```
(1, 2, 3, 4, 5, 6)
```

Repetition

```
t = ("Python",)
print(t * 3)
```

Output:

```
('Python', 'Python', 'Python')
```

Nested Tuple

```
data = (
    (1, 2),
    (3, 4),
    (5, 6)
)

print(data[1][0])
```

Output:

```
3
```

Useful Functions with Tuple

```
numbers = (10, 20, 30, 40)

print(len(numbers))
print(max(numbers))
print(min(numbers))
print(sum(numbers))
```

Output:

```
4
40
10
100
```

List vs Tuple

List	Tuple
[]	()
Mutable	Immutable
More methods	Only count() and index()

List	Tuple
Slower	Faster
More memory	Less memory

Example

```
my_list = [1, 2, 3]
my_tuple = (1, 2, 3)
```

Why Use Tuples?

Use tuples when data should not change.

Examples:

- Coordinates (x, y)
 - RGB colors (255, 0, 0)
 - Student records
 - Database results
-

Interview Questions

1. Difference between List and Tuple?

- List → Mutable
- Tuple → Immutable

2. Can we modify a tuple?

No, tuples are immutable.

3. How many methods does a tuple have?

Only 2 methods:

- `count()`
- `index()`

4. How to create a single-element tuple?

```
t = (10,)
```

Practice Programs

Print all elements of a tuple

```
t = (10, 20, 30, 40)

for i in t:
    print(i)
```

Find the sum of tuple elements

```
t = (10, 20, 30, 40)

print(sum(t))
```

Find the largest element

```
t = (10, 50, 20, 30)

print(max(t))
```

Summary

Create Tuple	-> ()
Access Element	-> tuple[index]
Slice Tuple	-> tuple[start:stop]
Count Value	-> count()
Find Index	-> index()
Length	-> len()
Maximum	-> max()
Minimum	-> min()
Sum	-> sum()
Check Element	-> in

Remember: List = Changeable ([]), Tuple = Unchangeable (()). This is the most important concept in tuples.

Link : [CHATGPT](#)

Dictionary in Python (Detailed Explanation)

A **Dictionary** is a built-in data structure in Python used to store data in **key-value pairs**.

Think of it like an English dictionary:

Word		Meaning
Apple	→	A fruit
Book	→	A collection of pages

In Python:

```
student = {  
    "name": "Krish",  
    "age": 20,  
    "city": "Bhavnagar"  
}
```

Here:

- "name", "age", "city" are **keys**
 - "Krish", 20, "Bhavnagar" are **values**
-

Characteristics of Dictionary

- ✓ Stores data as **key : value** pairs
 - ✓ Mutable (can be changed)
 - ✓ Ordered (Python 3.7+)
 - ✓ Keys must be unique
 - ✓ Values can be duplicated
 - ✓ Can store different data types
-

Creating a Dictionary

Method 1: Using Curly Braces {}

```
student = {  
    "name": "Krish",  
    "age": 20,  
    "city": "Bhavnagar"  
}
```

```
print(student)
```

Output

```
{'name': 'Krish', 'age': 20, 'city': 'Bhavnagar'}
```

Method 2: Using `dict()`

```
student = dict(name="Krish", age=20, city="Bhavnagar")
```

```
print(student)
```

Output

```
{'name': 'Krish', 'age': 20, 'city': 'Bhavnagar'}
```

Accessing Dictionary Values

Using Square Brackets `[]`

```
student = {  
    "name": "Krish",  
    "age": 20  
}  
  
print(student["name"])
```

Output

```
Krish
```

Using `get()`

```
print(student.get("age"))
```

Output

```
20
```

Difference Between `[]` and `get()`

```
student = {"name": "Krish"}  
  
print(student["age"])
```

Output:

KeyError

But:

```
print(student.get("age"))
```

Output:

None

You can also provide a default value:

```
print(student.get("age", "Not Found"))
```

Output:

Not Found

Adding Elements

```
student = {  
    "name": "Krish"  
}  
  
student["age"] = 20  
  
print(student)
```

Output

```
{'name': 'Krish', 'age': 20}
```

Updating Values

```
student = {  
    "name": "Krish",  
    "age": 20  
}  
  
student["age"] = 21  
  
print(student)
```

Output

```
{'name': 'Krish', 'age': 21}
```

Removing Elements

1. `pop()`

Removes the specified key.

```
student = {  
    "name": "Krish",  
    "age": 20  
}  
  
student.pop("age")  
  
print(student)
```

Output:

```
{'name': 'Krish'}
```

2. `popitem()`

Removes the last inserted item.

```
student = {  
    "name": "Krish",  
    "age": 20  
}  
  
student.popitem()  
  
print(student)
```

Output:

```
{'name': 'Krish'}
```

3. `del`

```
student = {  
    "name": "Krish",  
    "age": 20  
}  
  
del student["age"]  
  
print(student)
```

Output:

```
{'name': 'Krish'}
```

4. `clear()`

Removes all items.

```
student.clear()  
print(student)
```

Output:

```
{}
```

Dictionary Methods

Method	Description
<code>get()</code>	Returns value of key
<code>keys()</code>	Returns all keys
<code>values()</code>	Returns all values
<code>items()</code>	Returns key-value pairs
<code>update()</code>	Updates dictionary
<code>pop()</code>	Removes specified key
<code>popitem()</code>	Removes last item
<code>clear()</code>	Removes all items
<code>copy()</code>	Copies dictionary

`keys()`

```
student = {  
    "name": "Krish",  
    "age": 20  
}  
  
print(student.keys())
```

Output:

```
dict_keys(['name', 'age'])
```

`values()`

```
print(student.values())
```

Output:

```
dict_values(['Krish', 20])
```

items()

Returns both keys and values.

```
print(student.items())
```

Output:

```
dict_items([('name', 'Krish'), ('age', 20)])
```

Looping Through a Dictionary

Loop Through Keys

```
student = {  
    "name": "Krish",  
    "age": 20  
}
```

```
for key in student:  
    print(key)
```

Output:

```
name  
age
```

Loop Through Values

```
for value in student.values():  
    print(value)
```

Output:

```
Krish  
20
```

Loop Through Key-Value Pairs

```
for key, value in student.items():  
    print(key, ":", value)
```

Output:

```
name : Krish  
age : 20
```

Copying a Dictionary

```
student = {  
    "name": "Krish",  
    "age": 20  
}  
  
new_student = student.copy()  
  
print(new_student)
```

Output:

```
{'name': 'Krish', 'age': 20}
```

Nested Dictionary

A dictionary inside another dictionary.

```
students = {  
    "student1": {  
        "name": "Krish",  
        "age": 20  
    },  
    "student2": {  
        "name": "Rahul",  
        "age": 21  
    }  
}  
  
print(students["student1"]["name"])
```

Output:

```
Krish
```

Dictionary Comprehension

Similar to list comprehension.

Syntax

```
new_dict = {key: value for item in iterable}
```

Example: Squares

```
squares = {x: x**2 for x in range(1, 6)}  
  
print(squares)
```

Output:

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Membership Operators

```
student = {  
    "name": "Krish",  
    "age": 20  
}  
  
print("name" in student)  
print("city" in student)
```

Output:

```
True  
False
```

Useful Built-in Functions

```
student = {  
    "name": "Krish",  
    "age": 20,  
    "city": "Bhavnagar"  
}  
  
print(len(student))
```

Output:

```
3
```

Dictionary vs List vs Tuple

Feature	Dictionary	List	Tuple
Syntax	{ }	[]	()
Stores	Key-Value Pairs	Values	Values
Ordered	Yes	Yes	Yes
Mutable	Yes	Yes	No

Feature	Dictionary	List	Tuple
Duplicate Keys	No	Yes	Yes
Access By	Key	Index	Index

Real-Life Example

```
employee = {
    "id": 101,
    "name": "Krish",
    "department": "Data Science",
    "salary": 50000
}

print("Employee Name:", employee["name"])
print("Salary:", employee["salary"])
```

Output:

```
Employee Name: Krish
Salary: 50000
```

Interview Questions

1. Can dictionary keys be duplicated?

 No. Keys must be unique.

```
d = {
    "a": 1,
    "a": 2
}

print(d)
```

Output:

```
{'a': 2}
```

The latest value replaces the previous one.

2. Can dictionary values be duplicated?

 Yes.

```
d = {
    "a": 10,
```

```
"b": 10
}
```

3. Can lists be dictionary keys?

✗ No, because lists are mutable.

```
d = {
    [1, 2]: "Hello"
}
```

Output:

```
TypeError: unhashable type: 'list'
```

4. What types can be dictionary keys?

✓ Immutable types such as:

- String
- Integer
- Float
- Tuple

Example:

```
d = {
    (1, 2): "Tuple Key",
    1: "Integer Key",
    "name": "String Key"
}
```

Summary

Create Dictionary	-> {}
Access Value	-> d[key], d.get()
Add/Update	-> d[key] = value
Remove	-> pop(), popitem(), del, clear()
Get Keys	-> keys()
Get Values	-> values()
Get Items	-> items()
Loop	-> for key, value in d.items()
Copy	-> copy()
Nested Dictionary	-> Dictionary inside dictionary
Comprehension	-> {k:v for ...}

Example Program

```
student = {
```

```
        "name": "Krish",
        "age": 20
    }

    student["city"] = "Bhavnagar"

    for key, value in student.items():
        print(key, ":", value)
```

Output

```
name : Krish
age : 20
city : Bhavnagar
```

Important Points to Remember

- **Dictionary = Key : Value pairs**
- **Keys must be unique**
- **Values can be duplicated**
- **Dictionaries are mutable**
- **Use `get()` to avoid `KeyError`**
- **`items()` is commonly used in loops**

Link : [CHATGPT](#)

Link : [ALL](#)